

3.2_SJF

```
#include<stdio.h>

int main()
{
    int bt[10],rt[10],at[10],wt[10],tat[10];
    int n,i,time=0,remain,smallest,end;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        printf("Arrival and Burst time P%d: ",i+1);
        scanf("%d%d",&at[i],&bt[i]);
        rt[i]=bt[i];
    }

    remain=n;

    while(remain!=0)
    {
        smallest=-1;

        for(i=0;i<n;i++)
        {
            if(at[i]≤time && rt[i]>0)
            {
                if(smallest=-1 || rt[i]<rt[smallest])
                    smallest=i;
            }
        }

        if(smallest=-1)
        {
            time++;
            continue;
        }

        rt[smallest]--;
        time++;

        if(rt[smallest]=0)
        {
            remain--;
        }
    }
}
```

```

        end=time;

        tat[smallest]=end-at[smallest];
        wt[smallest]=tat[smallest]-bt[smallest];
    }
}

printf("\nP\tAT\tBT\tWT\tTAT\n");

for(i=0;i<n;i++)
    printf("P%d\t%d\t%d\t%d\t%d\n",
        i+1,at[i],bt[i],wt[i],tat[i]);

return 0;
}

```

3.3 Priority

```

#include<stdio.h>
int main()
{
    int bt[10],rt[10],at[10],pr[10],wt[10],tat[10];
    int n,i,time=0,remain,highest,end;
    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        printf("Arrival Burst Priority P%d: ",i+1);
        scanf("%d%d%d",&at[i],&bt[i],&pr[i]);
        rt[i]=bt[i];
    }

    remain=n;
    while(remain!=0)
    {
        highest=-1;

        for(i=0;i<n;i++)
        {
            if(at[i]≤time && rt[i]>0)
            {
                if(highest=-1 || pr[i]<pr[highest])

```

```

        highest=i;
    }
}

if(highest=-1)
{
    time++;
    continue;
}

rt[highest]--;
time++;

if(rt[highest]=0)
{
    remain--;
    end=time;

    tat[highest]=end-at[highest];
    wt[highest]=tat[highest]-bt[highest];
}
}

printf("\nP\tAT\tBT\tPR\tWT\tTAT\n");

for(i=0;i<n;i++)
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1,at[i],bt[i],pr[i],wt[i],tat[i]);

return 0;
}

```

BANKERS

```

#include<stdio.h>
int main()
{
    int n,m,i,j,k;
    int alloc[10][10], max[10][10], need[10][10];
    int avail[10], finish[10]={0}, safe[10];

    printf("Enter number of processes: ");
    scanf("%d",&n);

```

```

printf("Enter number of resources: ");
scanf("%d",&m);

printf("Enter Allocation matrix:\n");
for(i=0;i<n;i++)
    for(j=0;j<m;j++)
        scanf("%d",&alloc[i][j]);

printf("Enter Max matrix:\n");
for(i=0;i<n;i++)
    for(j=0;j<m;j++)
        scanf("%d",&max[i][j]);

printf("Enter Available resources:\n");
for(i=0;i<m;i++)
    scanf("%d",&avail[i]);

/* calculate Need */
for(i=0;i<n;i++)
    for(j=0;j<m;j++)
        need[i][j] = max[i][j] - alloc[i][j];

int count=0;

while(count<n)
{
    for(i=0;i<n;i++)
    {
        if(finish[i]==0)
        {
            int flag=0;

            for(j=0;j<m;j++)
            {
                if(need[i][j] > avail[j])
                {
                    flag=1;
                    break;
                }
            }

            if(flag==0)
            {
                for(k=0;k<m;k++)
                    avail[k] += alloc[i][k];
            }
        }
    }
    count++;
}

```

```

        safe[count]=i;
        finish[i]=1;
        count++;
    }
}

}

printf("\nSafe sequence:\n");

for(i=0;i<n;i++)
    printf("P%d ",safe[i]);

}

```

Memory Allocator

```

#include<stdio.h>

int main()
{
    int block[10],process[10];
    int bcopy[10];
    int m,n,i,j,choice;

    printf("Enter number of memory blocks: ");
    scanf("%d",&m);

    printf("Enter block sizes:\n");
    for(i=0;i<m;i++)
        scanf("%d",&block[i]);

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter process sizes:\n");
    for(i=0;i<n;i++)
        scanf("%d",&process[i]);

    do
    {
        for(i=0;i<m;i++)
            bcopy[i]=block[i];

        printf("\n--- MEMORY ALLOCATION MENU ---\n");
    }
}

```

```

printf("1. First Fit\n");
printf("2. Best Fit\n");
printf("3. Worst Fit\n");
printf("4. Exit\n");
printf("Enter choice: ");
scanf("%d",&choice);

if(choice==1)    /* FIRST FIT */
{
    printf("\nFirst Fit Allocation:\n");

    for(i=0;i<n;i++)
    {
        for(j=0;j<m;j++)
        {
            if(bcopy[j] ≥ process[i])
            {
                printf("Process %d → Block
%d\n",i+1,j+1);

                bcopy[j] -= process[i];
                break;
            }
        }
        if(j==m)
            printf("Process %d → Not Allocated\n",i+1);
    }
}

else if(choice==2)    /* BEST FIT */
{
    int best;

    printf("\nBest Fit Allocation:\n");

    for(i=0;i<n;i++)
    {
        best=-1;

        for(j=0;j<m;j++)
        {
            if(bcopy[j] ≥ process[i])
            {
                if(best=-1 || bcopy[j] < bcopy[best])
                    best=j;
            }
        }
    }
}

```

```

        if(best≠-1)
        {
            printf("Process %d → Block %d\n",i+1,best+1);
            bcopy[best]-=process[i];
        }
        else
            printf("Process %d → Not Allocated\n",i+1);
    }
}

else if(choice=3) /* WORST FIT */
{
    int worst;

    printf("\nWorst Fit Allocation:\n");

    for(i=0;i<n;i++)
    {
        worst=-1;

        for(j=0;j<m;j++)
        {
            if(bcopy[j] ≥ process[i])
            {
                if(worst=-1 || bcopy[j] > bcopy[worst])
                    worst=j;
            }
        }

        if(worst≠-1)
        {
            printf("Process %d → Block
%d\n",i+1,worst+1);
            bcopy[worst]-=process[i];
        }
        else
            printf("Process %d → Not Allocated\n",i+1);
    }
}

} while(choice≠4);
}

```

PageReplacement

```
#include<stdio.h>

int pages[50], frames[10];
int n,f;

/* FIFO */
void fifo()
{
    int i,j,k=0,flag,fault=0;

    for(i=0;i<f;i++) frames[i]=-1;

    for(i=0;i<n;i++)
    {
        flag=0;
        for(j=0;j<f;j++)
            if(frames[j]==pages[i])
                flag=1;
        if(flag==0)
        {
            frames[k]=pages[i];
            k=(k+1)%f;
            fault++;
        }
    }

    printf("Page Faults = %d\n",fault);
}

/* LRU */
void lru()
{
    int time[10];
    int i,j,pos,min,flag,fault=0;

    for(i=0;i<f;i++){
        frames[i]=-1;
        time[i]=0;
    }

    for(i=0;i<n;i++){
        flag=0;
```



```

        for(j=0;j<f;j++)
            if(frames[j]==pages[i]){
                flag=1;
                time[j]=i;
            }

        if(flag==0)
        {
            min=time[0];
            pos=0;

            for(j=1;j<f;j++)
                if(time[j]<min)
                {
                    min=time[j];
                    pos=j;
                }

            frames[pos]=pages[i];
            time[pos]=i;
            fault++;
        }
    }

    printf("Page Faults = %d\n",fault);
}

/* LFU */
void lfu()
{
    int freq[10];
    int i,j,pos,min,flag,fault=0;

    for(i=0;i<f;i++)
    {
        frames[i]=-1;
        freq[i]=0;
    }

    for(i=0;i<n;i++)
    {
        flag=0;

        for(j=0;j<f;j++)
            if(frames[j]==pages[i])
            {

```

```

        flag=1;
        freq[j]++;
    }

    if(flag==0)
    {
        min=freq[0];
        pos=0;

        for(j=1;j<f;j++)
            if(freq[j]<min)
            {
                min=freq[j];
                pos=j;
            }

        frames[pos]=pages[i];
        freq[pos]=1;
        fault++;
    }
}

printf("Page Faults = %d\n",fault);
}

/* Optimal */
void optimal()
{
    int i,j,k,pos,flag,farthest,fault=0;

    for(i=0;i<f;i++) frames[i]=-1;

    for(i=0;i<n;i++)
    {
        flag=0;

        for(j=0;j<f;j++)
            if(frames[j]==pages[i])
                flag=1;

        if(flag==0)
        {
            farthest=-1;
            pos=0;

            for(j=0;j<f;j++)

```

```

        {
            int found=0;

            for(k=i+1;k<n;k++)
                if(frames[j]==pages[k])
                {
                    if(k>farthest)
                    {
                        farthest=k;
                        pos=j;
                    }
                    found=1;
                    break;
                }

            if(found==0)
            {
                pos=j;
                break;
            }
        }

        frames[pos]=pages[i];
        fault++;
    }
}

printf("Page Faults = %d\n",fault);
}

int main()
{
    int choice,i;

    printf("Enter number of pages: ");
    scanf("%d",&n);

    printf("Enter page reference string:\n");
    for(i=0;i<n;i++)
        scanf("%d",&pages[i]);

    printf("Enter number of frames: ");
    scanf("%d",&f);

    do
    {
        printf("\n--- PAGE REPLACEMENT MENU --- \n");

```

```
    printf("1. FIFO\n");
    printf("2. LRU\n");
    printf("3. LFU\n");
    printf("4. Optimal\n");
    printf("5. Exit\n");

    printf("Enter choice: ");
    scanf("%d",&choice);

    if(choice==1) fifo();
    else if(choice==2) lru();
    else if(choice==3) lfu();
    else if(choice==4) optimal();

}while(choice!=5);

return 0;
}
```
